

Making Safe C++ Happen

An approach to implementing safety

Document #	P3700R0
Date	2025-05-19
Targeted subgroups	EWG, SG21, SG23
Ship vehicle	C++29
Reply-to	Peter Bindels <dascandy@gmail.com>

§ 1 Introduction

C++ is a language that has a lineage going back to the 1970's, which has attempted to remain backward-compatible with as much C as it reasonably can. While this has huge benefits, it comes with the downside that any construct that was created 50 years ago is still likely valid C++, even though we universally agree by now that the construct in question is always a terrible idea. Compilers have added many flags to enable users to prevent them from having these constructs end up in an executable or in production, and the culture has grown in the direction of enabling more and more warnings. Still, the language retains the reputation that it's possible to write very unsafe code, and that it's common to do so.

Changing a language with a spread the size of C++ to become safe is a huge undertaking, and not one that can be done in a single, or even in a single dozen papers. This paper attempts to provide a structure to identify and to map progress on the area of safety in C++. It does not propose anything specific itself. In particular, contracts (P2900), profiles (p3081 et al) and others are likely ways to implement part of this.

§ 2 What is safety?

Taking a page from <https://wg21.link/p2687>'s book, we find the following set of safety failures. I've ordered them roughly from user responsibility to language responsibility

- Logic errors: perfectly legal constructs that don't reflect the programmer's intent, such as using < where a <= or a > was intended.
- Timing errors: for example, delivering a result in 1.2ms to a device supposedly responding to an external event in 1ms.

- Concurrency errors: failing to correctly take current activities into account leading to (typically) obscure problems (such as data races and deadlocks).
- Resource leaks: failing to return resources to their appropriate management system (e.g., memory, file handles, locks) potentially leading to the program grinding to a halt because of lack of available resources.
- Termination errors: a library that terminates in case of “unanticipated conditions” being part of a program that is not allowed to unconditionally terminate.
- Overflows and unanticipated conversions: For example, an unanticipated wraparound of an unsigned integer loop variable or a narrowing conversion.
- Memory corruption: for example, through the result of a range error or by accessing memory through a pointer to an object that no longer exists thereby changing a different object.
- Type errors: for example, using the result of an inappropriate cast or accessing a union through a member different from the one through which it was written.

For those at the bottom, the language is fully in a position to fix what it does but currently doesn't. For those at the top, the language is very much in a position where it is unable to do anything directly, but it still has some sway on giving the user tools to help themselves find these problems.

This paper covers termination errors, overflows and unanticipated conversions, type errors, and memory corruption. It offers hooks to handle situations like resource leaks, concurrency errors, logic errors and timing errors in a uniform way, but does not attempt to find or handle them.

§ 3 The requirements

These are aspects that we consider to be fundamental to any safety-in-c++ proposal to succeed. We are enumerating them here, so we can refer to them in resulting design properties.

- Code that works in the safe mode is guaranteed to work in the regular mode, and has the same outcomes
- Switching to the safe mode is incremental, allowing people to adopt it over time and as budget is available
- The safe mode is customizable; safety is not a universally-identical thing and people have different priorities in what safety is desired first
- Large scale software must remain supported; we can consider global guarantees but only if they can be done in a way that scales
- Failure in safety at runtime is handled in an application-appropriate way
- Applications using the safe mode are not slower than those using regular C++
- The expressivity of the language does not go down

§ 4 The approach

§ 4.1 Prevent at compile time what we can

- Types encode information about semantics and values. When the types cannot, annotations do
- Encode information on TU and function boundaries that translate global problems into local ones
- Disallow local constructs that are not safe

§ 4.2 Prevent at runtime what we cannot guarantee at compile time

- Emit runtime checks for any non-prevented safety failure
- Have a general approach to handle runtime failure that is suitable for all users
- Allow compilers to take what is specified as a runtime check, and to move it to compile time if they are able to do so

§ 4.3 Offer a user friendly interface to start using safer C++, and to increase the safety requirements

- Offer ways for end-users, other standards bodies, and the C++ committee, to define checks, language reductions, and groupings of related checks and reductions, and to enable those in parts of software
- Offer ways for end-users to incrementally adopt stricter practices (Similar to how warnings get switched into errors, and how users can enable more warnings to be fixed)
- Offer ways for users, library developers, end-product developers etc. to communicate with non-C++-developers about the safety of their C++ usage

§ 5 The grid

We propose a grid of things to do. The rows of the grid correspond to a type of action to do, a particular tool to use. The columns of the grid correspond to an area of safety that is being addressed, similar to [P3081](#)'s Profiles. For example, changing `dynamic_cast` on unrelated types to ill-formed would be in the "type-safety" column, in the "language subsetting" tool. Making `operator[]` on a `std::span` runtime fail on out-of-range access is in the "range safety" column with the "runtime checks" tool.

Existing papers can be mapped onto this grid by realizing that the columns roughly correspond to Profiles.

The rows are split up between separate papers, where each paper tackles a separate approach, specifying it in detail, but leaving the specific application in the language to other papers.

The approach we see that will make the grid a reality is to start by making sure we have all the tools that are needed to make a column implementable, then proposing each column in its own paper that tries to tackle that aspect of safety in a targeted way by using all tools in conjunction. As soon as the tools are specified, all other standard bodies can also do the same, allowing (for example) MISRA to define a MISRA-2023 profile, including compile-time and run-time checks.

§ 6 The rows

We need to have tools that enable us to enforce safety. The rows we identify so far are:

- We need the types and annotations that convey sufficient information for tools to prove safety, and to transfer knowledge of safety and values between translation units.
- We need to have a way to remove parts of the language that are now considered to be "bad code" or "un-removable in the general case", that is usable by end users and other standard bodies.
- We need to provide substitute functionality for essential expressivity that is removed
- We need to have a way to handle runtime failure in such a way that each kind of user is able to handle the runtime failures in a way appropriate for their product.
- We need to have a way to name a column, to make the mass of new restrictions and checks tractable for users and integrators to understand, for usability switches to be portable between systems, and for statements about safety to have a lingua franca.

For each row, it is likely that we need a conjunction of multiple tools to cover it fully.

§ 6.1 Cross-TU / cross-function information

Software is written with many assumptions. These are often encoded into the program itself by use of an `assume()` function or macro, which verifies the veracity of the assumptions in passing. The downside of adding many of these assumption-documenting notes is that they tend to interact badly with separate compilation. This leads to developers leaving out many such documented assumptions, making it harder for the compiler to prove that those assumptions are in fact tautological and optimizing it out. In particular, on translation unit boundaries, compilers get a known partial set of information, and can not use the

information inferrable from other functions in optimizing this translation unit. At best they can try to store the program code until link time and hope that enough information is available to make such conclusions then, and that the compiler is powerful enough to hold all information until that point.

In many places, having the ability to annotate on function boundaries what assumptions must hold upon entry (preconditions), which assumptions can be made on exit from a valid function call (postconditions), where ownership and lifetime of arguments go (pointer lifetimes) etc. would be a boon to allow the compiler to determine that a function by itself either does or does not adhere to its own assumptions. In an idealized compiler, this would be able to prove that a function is correct, or that it somewhere breaks its indicated conditions. In a practical compiler, it will likely allow compilers to warn on cross-TU constructs that are now detectably wrong on either side, and in some (many) cases, to optimize the code with the knowledge it's checked and enforced, leading to safer and faster programs.

Existing work was done in many papers adding ownership-containing and lifetime-transferring types (such as `unique_ptr`, `vector` etc.). We also include <https://wg21.link/p2900> giving users a way to cross-TU annotate functions with value expectations on entry and exit.

§ 6.2 Remove parts of the language

For safety, we want to remove unsafe constructs:

- In hard-embedded setups, we want to avoid dynamic allocations
- In cross-language environments, we want to avoid exceptions
- In "Modern C++", we want to avoid raw `new` and `delete` statements in user code, pointer arithmetic, use of raw arrays, and similar constructs
- Many people want to use the "without-C" subset of C++
- Library authors want to use the "C++17-compatible" subset
- MISRA and AutoSAR users want to use the compliant subset

Language subsetting is a fundamental thing that many people do all the time, in many different areas. To add safety, we need to have a way to define subsets.

§ 6.3 Provide substitutes for removed essential functionality

Many papers in WG21 already take care of providing new and safe ways to express functionality that before that point was only expressible with either disciplined use of less-structured approaches, or by skirting around things that were not quite correct but that functioned regardless. This row exists mostly to ensure that for each column we define, we realize that we may need to define new functionality to replace what we are removing, or alternatively that we should

clearly illustrate in each column paper what the safe replacements are for the constructs that are being removed.

§ 6.4 Handling runtime failure

Depending on the environment, we need a different way to handle failures. The environments vary from Voyager probes at 15 light-minutes distance from Earth, autonomous cars on public roads, server software handling billions of requests per minute, IoT devices handling five requests per day, implanted pace makers, phone software displaying a video stream, to high-frequency trading software and bank software.

The best way to handle a runtime safety failure for all of these depends greatly on what kind of software it is, what environment it is in, and what the software owner wants the handling to be. For some it will be collecting the errors and sending them as telemetry data to a backend; for some it is more important to get the device into a known-safe state; a third will have to keep working as best it can while sending out a "device needs help" signal and a fourth is best off just crashing to desktop. In some cases we even want different kinds of failure to have different kinds of treatment - for example, a newly added check that may fail, would be best off only being observed, while a check that has existed for months and that is known to be essential to prevent exploitation must be enforced.

Part of this row's efforts should also be remapping places where our current best handling is to call `std::terminate` directly, to instead direct to this handler with information on what exactly happened, so that it can figure out what way to handle the failure and to ensure the software keeps performing its job as well as it can.

§ 6.5 Naming safety

People tend not to use things that are hard to use, even if using them would be good. In order to make this accessible to people, we will have to enable people to switch on parts of safety at a time, so that they can incrementally adopt more safe standards while updating the code base, one step at a time. We also need to make it possible for people to enable well-known groupings or increments of safety, somewhat similar to current `-Wall` or MISRA-2023, in a clear, consistent and portable manner.

To do this, we need to have some way for people to group related settings, checks and properties, for end-users to activate, and for quality assurance to verify. It is much easier to speak about "turning on the MISRA-2023 profile" than a series of checks, even if the meaning is the same.

§ 7 The columns

The columns correspond to Profiles, and this paper will not attempt to copy the information already written on it.